

Temporal Difference (TD) – Model Predictive Control (MPC) algorithms

- TD-MPC
- TD-MPC2
- TD-M(PC)²

1. TD-MPC 첫 논문

모델예측제어 (Model predictive control, MPC)

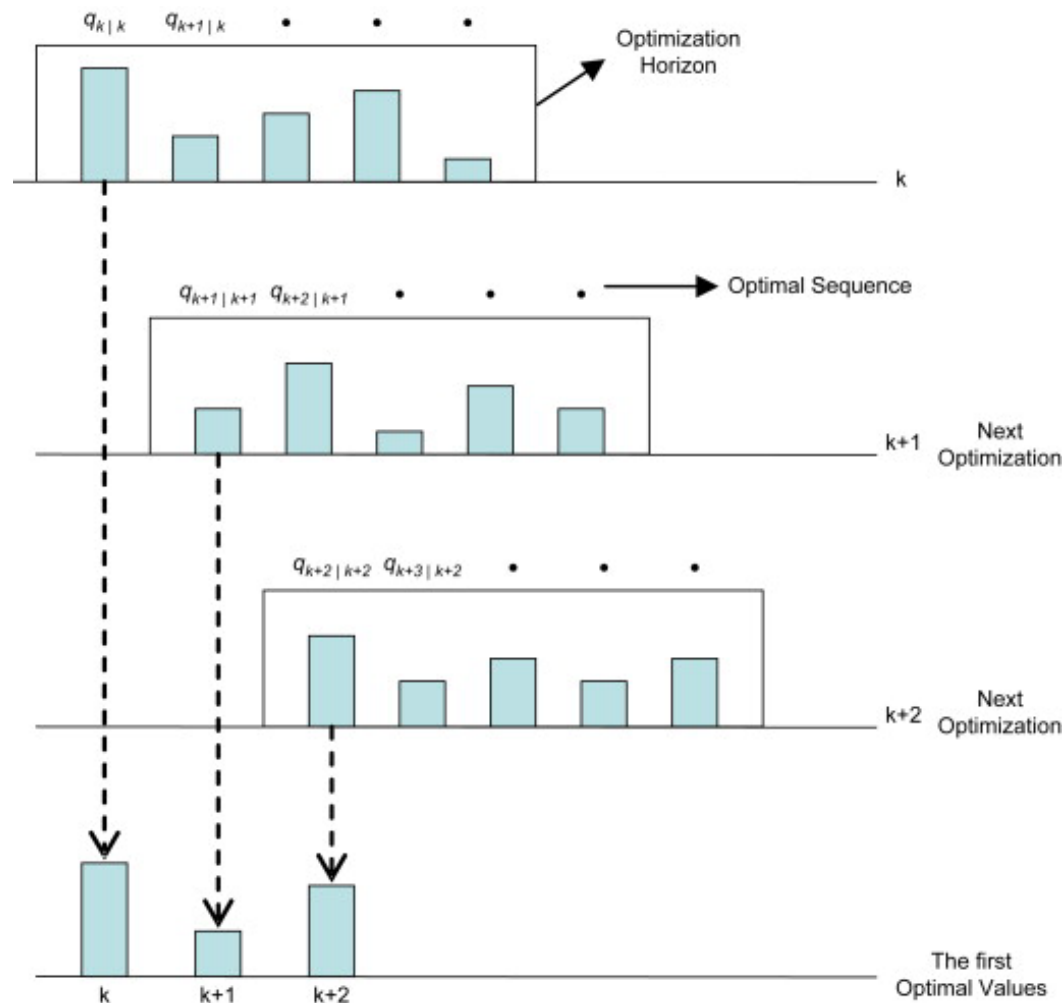
✓ Planning

- ✓ 모델에 기반한 최적화 문제를 풀어 **Policy** 를 결정

$$\Pi_{\theta}^{MPC}(s_t) = \operatorname{argmax}_{a_{\{t:t+H\}}} \mathbb{E}[\sum \gamma^i R(s_i, a_i)]$$

✓ Receding horizon

- ✓ 현재 상태 s_t 에 기반한 최적화 문제의 해
 - ✓ = Action trajectory $a_t: a_{t+H-1}$
- ✓ 그러나 **모델은 부정확**할 수 있음
- ✓ 또한 더 먼 미래의 action 은 **부정확도가 누적**됨
- ✓ 따라서 **첫 action a_t 만 쓰고**,
 - ✓ 시스템 변화를 다시 관측하여
 - ✓ 새로운 초기 상태 s_{t+1} 에 기반한 해를 재계산
 - ✓ 이러면 모델 부정확도의 누적을 피할 수 있음
- ✓ **그래도 정확한 모델은 여전히 중요함**



Model predictive Path integral (MPPI)

- ✓ 그럼 최적화 문제를 어떻게 푸느냐?

- ✓ Gradient descent 방식으로 목적 함수를 극대화
- ✓ 전통적으로 쉬운 모델 (선형, 2차)이 쓰임

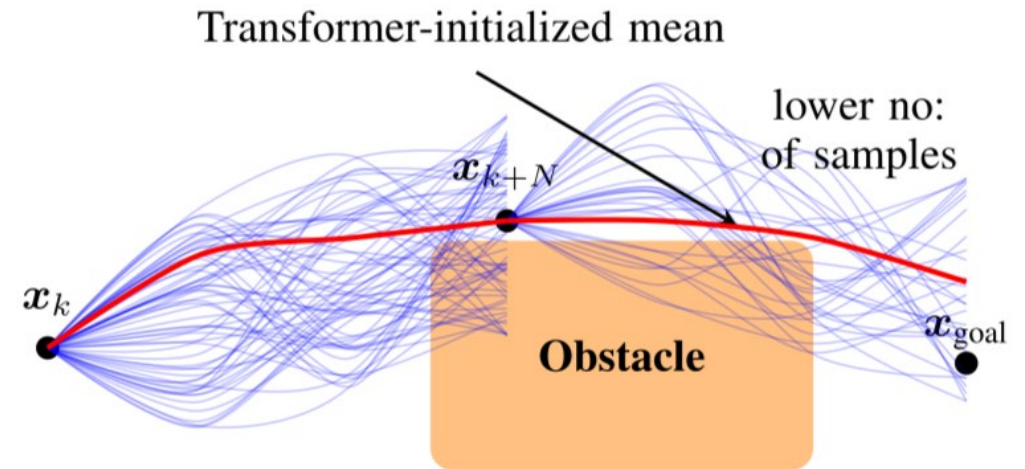
- ✓ 그런데 복잡한 모델을 사용한다면? (e.g, 파라미터 5M개)

- ✓ Gradient descent 계산이 어렵고
- ✓ 비선형이라 local minima에 빠질 가능성 높음
- ✓ 또한 gradient descent 의 double loop 가 생기게 됨
 - ✓ World model 학습 (=gradient descent)
 - ✓ World 모델 기반 planning (=gradient descent)

- ✓ 대안 -> **Sampling based 풀이법 (MPPI)**

- ✓ 무수히 많은 경로의 return을 평가하고,
- ✓ Return에 비례한 weighted sum을 해서 최적 경로를 구함
- ✓ (Feynman-Kac Lemma 참고)

$$\Pi_{\theta}^{MPC}(s_t) = \operatorname{argmax}_{a_{\{t:t+H\}}} \mathbb{E}[\sum \gamma^i R(s_i, a_i)]$$



Terminal value approximation

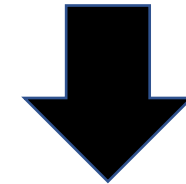
- ✓ 장기적인 Planning이 필요하다면?

- ✓ 짧은 Horizon H만큼만 planning을 하면, 근시안적인 정책을 얻게 됨 (Myopic policy)
- ✓ 하지만 Long horizon planning을 하면 최적화 계산량이 기하급수적으로 증가
- ✓ Scalable하지 않음...

- ✓ 흔히 쓰이는 대안 -> Terminal value approximation

- ✓ 적당한 horizon 만 planning을 하고,
- ✓ Horizon에서의 state의 value를 추정...
- ✓ 이를 활용해서 짧은 horizon planning 으로도, 근시안적이지 않은 정책을 얻을 수 있음

$$\Pi_{\theta}^{MPC}(s_t) = \operatorname{argmax}_{a_{\{t:t+H\}}} \mathbb{E} \left[\sum \gamma^i R(s_i, a_i) \right]$$



$$\Pi_{\theta}^{MPC}(s_t) = \arg \max_{\mathbf{a}_{t:t+H-1}} \mathbb{E} \left[\left(\sum_{i=0}^{H-1} \gamma^i R(s_{t+i}, a_{t+i}) \right) + \gamma^H V(s_{t+H}) \right]$$

Terminal value approximation

- ✓ Terminal state의 value를 알기 위해선 state value function V 가 필요

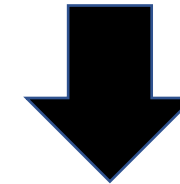
$$\Pi_{\theta}^{MPC}(s_t) = \operatorname{argmax}_{a_{\{t:t+H\}}} \mathbb{E} \left[\sum \gamma^i R(s_i, a_i) \right]$$

- ✓ 그러나 강화학습에서 State value function V 는 활용하기 쉽지 않음

- ✓ Direct하게 V 를 사용하는 방법론들은 sample 효율성이 떨어짐 (Policy gradient 계열 – TRPO, PPO 등)

- ✓ Replay buffer를 사용해서 Sample 효율성을 높이려면
 - ✓ **Q function (=State-action value) 활용 필요**

- ✓ Q function에 기반해서 V 를 구하려면...
 - ✓ Greedy policy 에서는 max operator 계산이 필요
 - ✓ 특히 Continuous action space에서는 max 계산이 엄청 어려움...



$$\Pi_{\theta}^{MPC}(s_t) = \arg \max_{\mathbf{a}_{t:t+H-1}} \mathbb{E} \left[\left(\sum_{i=0}^{H-1} \gamma^i R(s_{t+i}, a_{t+i}) \right) + \gamma^H V(s_{t+H}) \right]$$

$$V^*(s) = \max_a Q^*(s, a)$$

TD-MPC 알고리즘 0) 소개

- ✓ 특징 1) MPC planning 에서 World model 를 활용하기 위해 sampling based 방법론인 MPPI를 활용
- ✓ 특징 2) Planning horizon을 제한하면서도 근시안적인 planning 을 피하기 위해, Terminal state에 대한 value 근사 활용.
 - ✓ 다만 **Terminal V**를 얻는 것은 어렵기 때문에
 - ✓ **Terminal Q** Q_θ 와 **Policy prior** π_θ 로 근사
 - ✓ 이후 Q_θ 는 **Temporal difference** 방식으로 학습
- ✓ 특징 3) 목표에 불필요한 것들을 모델링에서 제외하기 위해, World model들을 Latent space에서 학습
 - ✓ Latent embedding h_θ
 - ✓ Env dynamics d_θ
 - ✓ Reward R_θ
 - ✓ Terminal Q function

$$\Pi_\theta^{\text{MPC}}(s_t) = \arg \max_{\mathbf{a}_{t:t+H-1}} \mathbb{E} \left[\left(\sum_{i=0}^{H-1} \gamma^i R(s_{t+i}, \mathbf{a}_{t+i}) \right) + \gamma^H V(s_{t+H}) \right]$$

$$V(s) = \max_a Q(s, a) \cong Q(s, \pi_\theta(s))$$

$$\mathcal{L}(\theta; \Gamma_i) = c_1 \underbrace{\|R_\theta(\mathbf{z}_i, \mathbf{a}_i) - r_i\|_2^2}_{\text{reward}} \quad (8)$$

$$+ c_2 \underbrace{\|Q_\theta(\mathbf{z}_i, \mathbf{a}_i) - (r_i + \gamma Q_\theta(\mathbf{z}_{i+1}, \pi_\theta(\mathbf{z}_{i+1})))\|_2^2}_{\text{value}} \quad (9)$$

$$+ c_3 \underbrace{\|d_\theta(\mathbf{z}_i, \mathbf{a}_i) - h_\theta(\mathbf{s}_{i+1})\|_2^2}_{\text{latent state consistency}} \quad (10)$$

Representation:	$\mathbf{z}_t = h_\theta(\mathbf{s}_t)$
Latent dynamics:	$\mathbf{z}_{t+1} = d_\theta(\mathbf{z}_t, \mathbf{a}_t)$
Reward:	$\hat{r}_t = R_\theta(\mathbf{z}_t, \mathbf{a}_t)$
Value:	$\hat{q}_t = Q_\theta(\mathbf{z}_t, \mathbf{a}_t)$
Policy:	$\hat{\mathbf{a}}_t \sim \pi_\theta(\mathbf{z}_t)$

TD-MPC 알고리즘 1) Planning (MPPI)

Algorithm 1 TD-MPC (*inference*)

Require: θ : learned network parameters

μ^0, σ^0 : initial parameters for $\mathcal{N}$

N, N_π : num sample/policy trajectories

$\mathbf{s}_t, H$: current state, rollout horizon

1: Encode state $\mathbf{z}_t \leftarrow h_\theta(\mathbf{s}_t)$ $\triangleleft$ Assuming TOLD model

2: **for** each iteration $j = 1..J$ **do**

3: Sample N traj. of len. H from $\mathcal{N}(\mu^{j-1}, (\sigma^{j-1})^2 \mathbf{I})$

4: Sample N_π traj. of length H using π_θ, d_θ

// Estimate trajectory returns ϕ_Γ using $d_\theta, R_\theta, Q_\theta$,
starting from $\mathbf{z}_t$ and initially letting $\phi_\Gamma = 0$:

5: **for** all $N + N_\pi$ trajectories $(\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H})$ **do**

6: **for** step $t = 0..H - 1$ **do**

7: $\phi_\Gamma = \phi_\Gamma + \gamma^t R_\theta(\mathbf{z}_t, \mathbf{a}_t)$ $\triangleleft$ Reward

8: $\mathbf{z}_{t+1} \leftarrow d_\theta(\mathbf{z}_t, \mathbf{a}_t)$ $\triangleleft$ Latent transition

9: $\phi_\Gamma = \phi_\Gamma + \gamma^H Q_\theta(\mathbf{z}_H, \mathbf{a}_H)$ $\triangleleft$ Terminal value

// Update parameters μ, σ for next iteration:

10: $\mu^j, \sigma^j = \text{Equation 4 (and Equation 5)}$

11: **return** $\mathbf{a} \sim \mathcal{N}(\mu^J, (\sigma^J)^2 \mathbf{I})$

✓ 현재 state s_0 에서...

1) 길이 H 의 action trajectory $a_0, \dots, a_{H-1}$ 샘플링

2) 각 Action trajectory 의 return 을 추정

✓ 2-1) World model R_θ, d_θ 를 이용,

✓ s_0 에 $a_0, \dots, a_{H-1}$ 를 적용할 경우

✓ 예측되는 discounted return 를 합산

✓ 결과적으로 도달할 state s_H 를 예측

✓ 2-2) World model Q_θ, p_θ 를 이용,

✓ s_H 의 value를 추가

3) Return이 높은 Action trajectory를 뽑을 수 있도록
Action sampling distribution $\mathcal{N}(\mu, \sigma)$ update

✓ 이렇게 뽑힌 action으로 환경과 상호작용

✓ "Behavior policy"

TD-MPC 알고리즘 2) Training

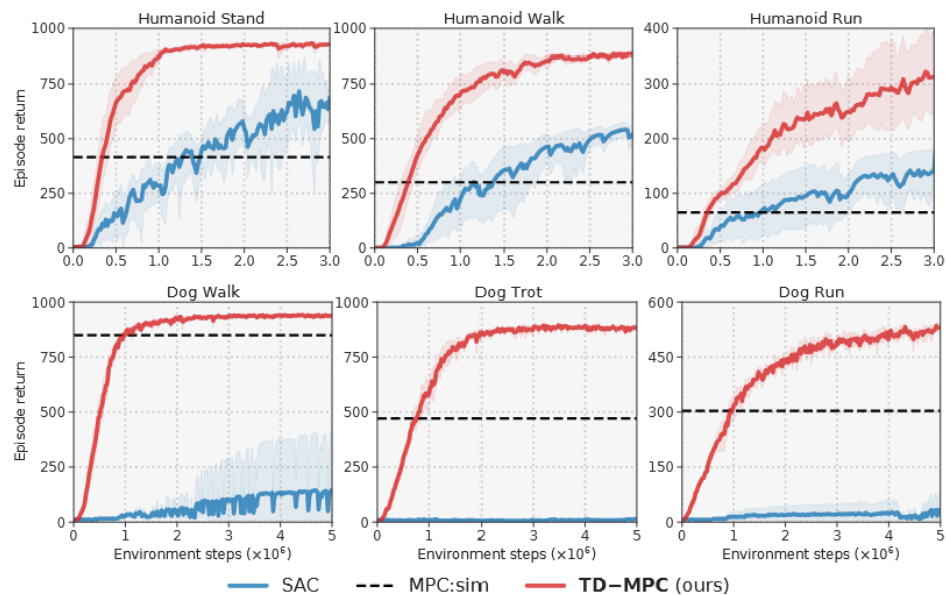
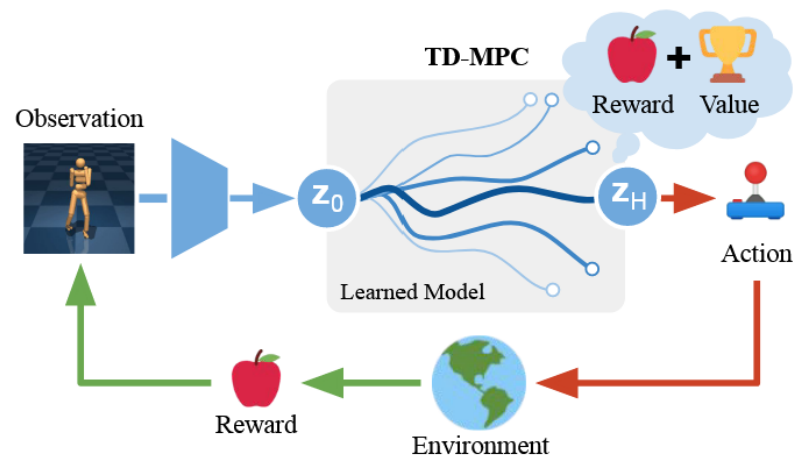
$$\mathcal{J}(\theta; \Gamma) = \sum_{i=t}^{t+H} \lambda^{i-t} \mathcal{L}(\theta; \Gamma_i)$$

- ✓ Buffer는 Trajectory $\Gamma = (s_t, a_t, r_t, s_{t+1})_{t:t+H}$ 를 저장
- ✓ Multiple roll-out steps에 대해 loss 계산

$$\begin{aligned} \mathcal{L}(\theta; \Gamma_i) = & c_1 \underbrace{\|R_\theta(\mathbf{z}_i, \mathbf{a}_i) - r_i\|_2^2}_{\text{reward}} \\ & + c_2 \underbrace{\|Q_\theta(\mathbf{z}_i, \mathbf{a}_i) - (r_i + \gamma Q_\theta(\mathbf{z}_{i+1}, \pi_\theta(\mathbf{z}_{i+1})))\|_2^2}_{\text{value}} \\ & + c_3 \underbrace{\|d_\theta(\mathbf{z}_i, \mathbf{a}_i) - h_\theta(\mathbf{s}_{i+1})\|_2^2}_{\text{latent state consistency}} \end{aligned}$$

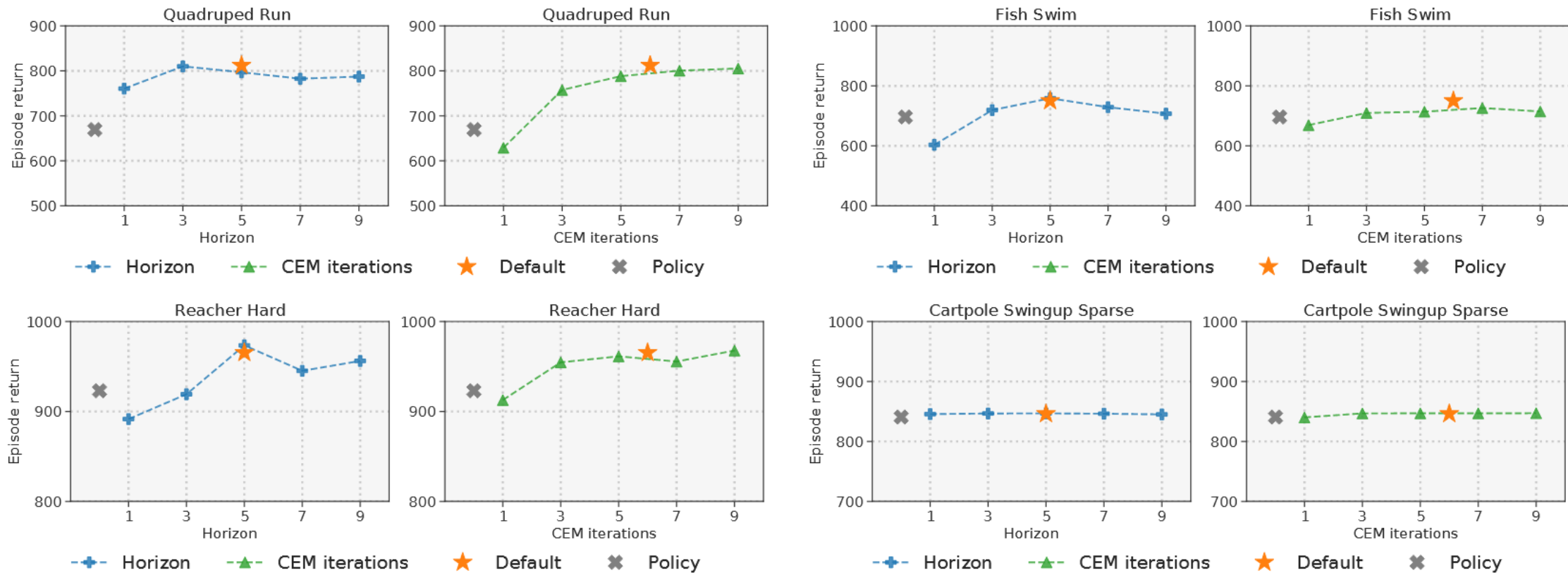
- ✓ 각 Trajectory의 loss는 3가지 term을 고려
 - ✓ 1) Reward 예측이 맞는지
 - ✓ 2) Terminal value 예측이 맞는지
 - ✓ $Q_\theta(z_i, a_i)$ 는 planning policy의 action
 - ✓ $Q_\theta(z_{i+1}, \pi_\theta(z_{i+1}))$ 은 policy prior p_θ 의 action
 - ✓ 3) Consistency term
 - ✓ Latent dynamics 예측이
 - ✓ 환경의 다음 state s_{t+1} 를 잘 예측하는지

TD-MPC 알고리즘 3) 성능



- ✓ SAC를 outperform
- ✓ 기존 Continuous action space SOTA

TD-MPC 알고리즘 3) 성능



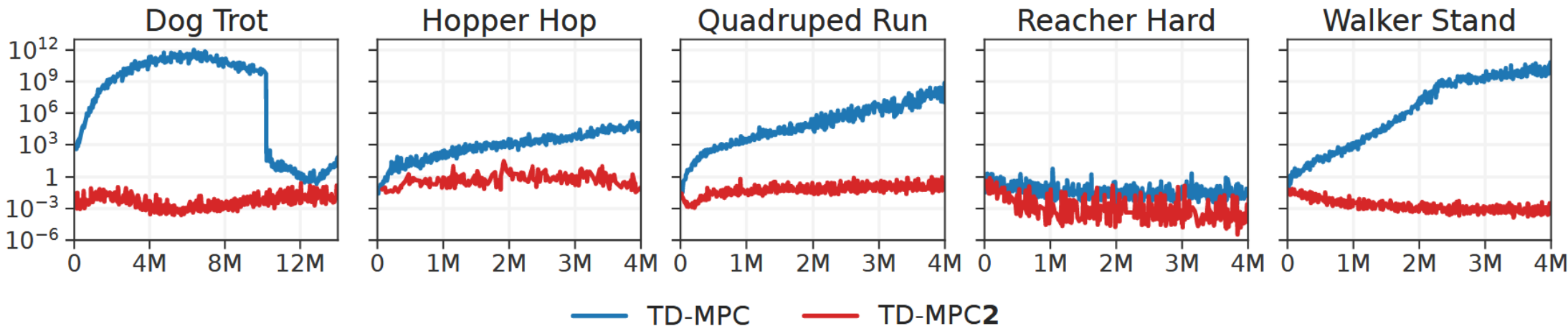
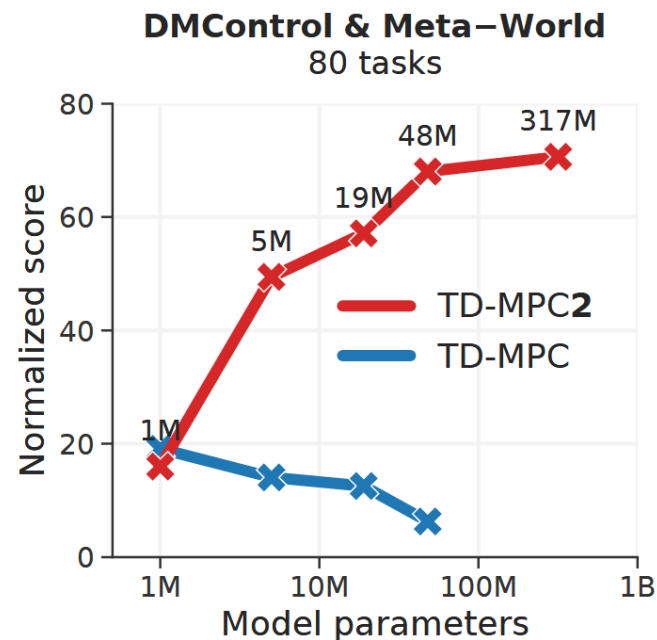
- ✓ (x marker) Terminal Q function 만 학습한 Policy prior p_{θ} 도 준수한 성능을 보임
- ✓ Horizon 증가에 따라 대체적으로 성능 증가 보이지만, 고난이도 task에 대해서는 한계 보임

2. TD-MPC2

Multi-task generalist를 위한 개선

TD-MPC의 문제...

- ✓ (Scalability) Parameter 수가 증가해도 성능이 비례하지 않음
 - ✓ 강화학습 알고리즘들에 공통적으로 나타나는 문제!
 - ✓ 학습 과정에서 **Gradient Norm**이 발산하는 현상
 - ✓ Normalization 필요!!!
- ✓ 진정한 **Multi-task generalist**가 아님
 - ✓ 하이퍼 파라미터 너무 타고...
 - ✓ Task 난이도에 따라 성능 차이가 너무 큼



TD-MPC2의 개선 방안 1)

- ✓ 문제) (**Scalability**) Parameter 수가 증가해도 성능이 비례하지 않음
 - ✓ 강화학습 알고리즘들에 공통적으로 나타나는 문제!
 - ✓ 학습 과정에서 **Gradient Norm**이 발산하는 현상
 - ✓ 하이퍼 파라미터도 너무 타고...
- ✓ 개선 1) 각종 Normalization 도입
 - ✓ (SimNorm) Latent representation를 Soft max 방식으로 Normalize
 - ✓ 기타 LayerNorm, Mish, Q ensemble 도입
- ✓ 개선 2) Reward / Terminal value 예측 모델의 discretization
 - ✓ **Reward/Terminal Q 모델을 각 bin들의 확률 예측 방식으로 변경**
 - ✓ Log-space에서 discretization
 - ✓ **Cross entropy loss로 변경**
 - ✓ Reward의 scale에 민감한 문제 해결
- ✓ 개선 3) Policy prior에 Maximum entropy objective 도입
 - ✓ 탐색 과정의 하이퍼파라미터 민감성을 낮춤

Representation:	$\mathbf{z}_t = h_\theta(\mathbf{s}_t)$
Latent dynamics:	$\mathbf{z}_{t+1} = d_\theta(\mathbf{z}_t, \mathbf{a}_t)$
Reward:	$\hat{r}_t = R_\theta(\mathbf{z}_t, \mathbf{a}_t)$
Value:	$\hat{q}_t = Q_\theta(\mathbf{z}_t, \mathbf{a}_t)$
Policy:	$\hat{\mathbf{a}}_t \sim \pi_\theta(\mathbf{z}_t)$

$$\mathcal{L}(\theta; \Gamma_i) = c_1 \underbrace{\|R_\theta(\mathbf{z}_i, \mathbf{a}_i) - r_i\|_2^2}_{\text{reward}} \quad (8)$$

$$+ c_2 \underbrace{\|Q_\theta(\mathbf{z}_i, \mathbf{a}_i) - (r_i + \gamma Q_\theta(\mathbf{z}_{i+1}, \pi_\theta(\mathbf{z}_{i+1})))\|_2^2}_{\text{value}} \quad (9)$$

$$+ c_3 \underbrace{\|d_\theta(\mathbf{z}_i, \mathbf{a}_i) - h_\theta(\mathbf{s}_{i+1})\|_2^2}_{\text{latent state consistency}} \quad (10)$$



$$\mathcal{L}(\theta) \doteq \mathbb{E}_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}')_{0:H} \sim \mathcal{B}} \left[\sum_{t=0}^H \lambda^t \left(\underbrace{\|\mathbf{z}'_t - \text{sg}(h(\mathbf{s}'_t))\|_2^2}_{\text{Joint-embedding prediction}} + \underbrace{\text{CE}(\hat{r}_t, r_t)}_{\text{Reward prediction}} + \underbrace{\text{CE}(\hat{q}_t, q_t)}_{\text{Value prediction}} \right) \right]$$

TD-MPC2의 개선 방안 2)

- ✓ 문제) 진정한 **Multi-task generalist**가 아님
 - ✓ 하이퍼 파라미터 너무 타고...
 - ✓ Task 난이도에 따라 성능 차이가 너무 큼
- ✓ 개선) **Task embedding** 개념 도입
 - ✓ Task similarity를 exploit할 수 있도록, 공통적인 representation 을 학습
 - ✓ 비슷한 task -> 비슷한 학습

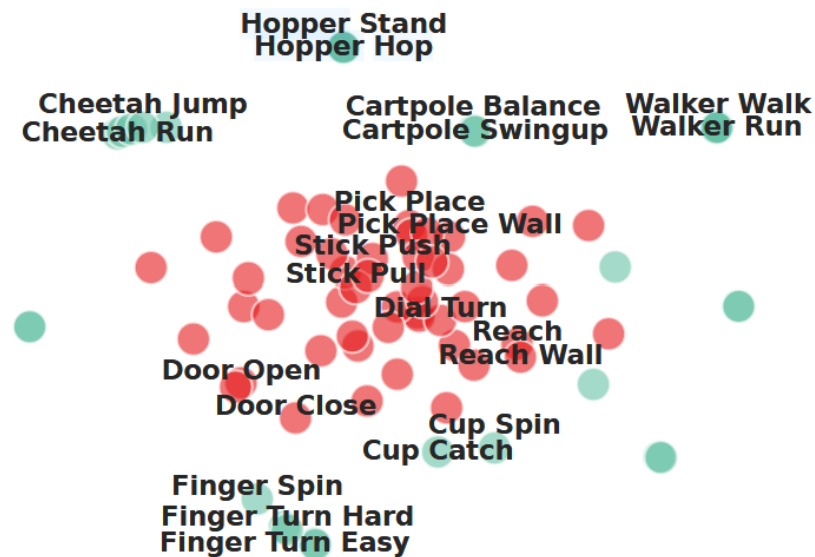
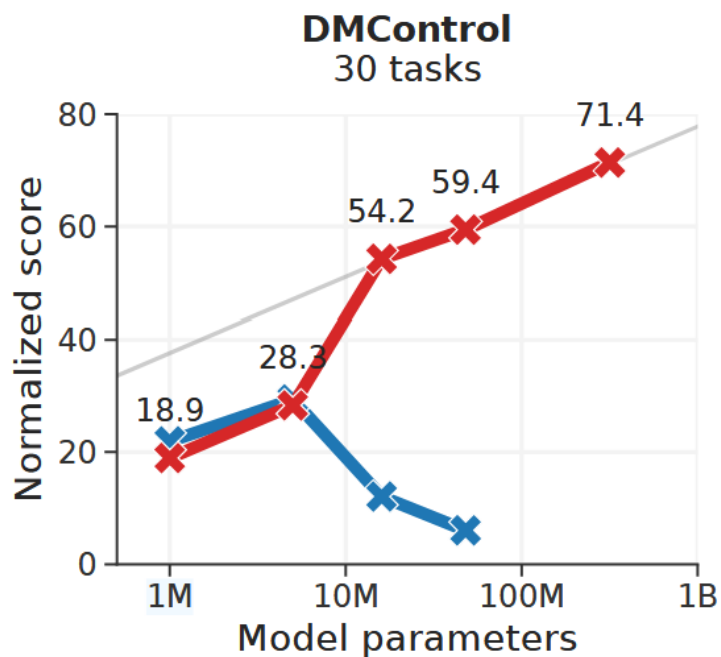
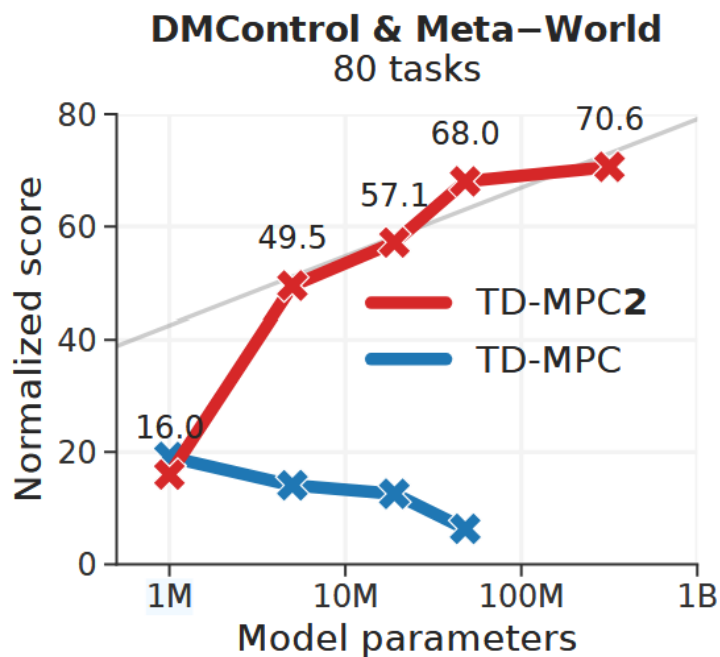
Representation:	$\mathbf{z}_t = h_\theta(\mathbf{s}_t)$
Latent dynamics:	$\mathbf{z}_{t+1} = d_\theta(\mathbf{z}_t, \mathbf{a}_t)$
Reward:	$\hat{r}_t = R_\theta(\mathbf{z}_t, \mathbf{a}_t)$
Value:	$\hat{q}_t = Q_\theta(\mathbf{z}_t, \mathbf{a}_t)$
Policy:	$\hat{\mathbf{a}}_t \sim \pi_\theta(\mathbf{z}_t)$



Encoder	$\mathbf{z} = h(\mathbf{s}, \mathbf{e})$
Latent dynamics	$\mathbf{z}' = d(\mathbf{z}, \mathbf{a}, \mathbf{e})$
Reward	$\hat{r} = R(\mathbf{z}, \mathbf{a}, \mathbf{e})$
Terminal value	$\hat{q} = Q(\mathbf{z}, \mathbf{a}, \mathbf{e})$
Policy prior	$\hat{\mathbf{a}} = p(\mathbf{z}, \mathbf{e})$

TD-MPC2의 개선 결과

- ✓ Scalability 달성!!
- ✓ Task embedding 결과 -> 목적이 아니라, **dynamics가 비슷한 task**끼리 묶이는 경향



3. TD-MPC with policy constraint (TD-M(PC)²)

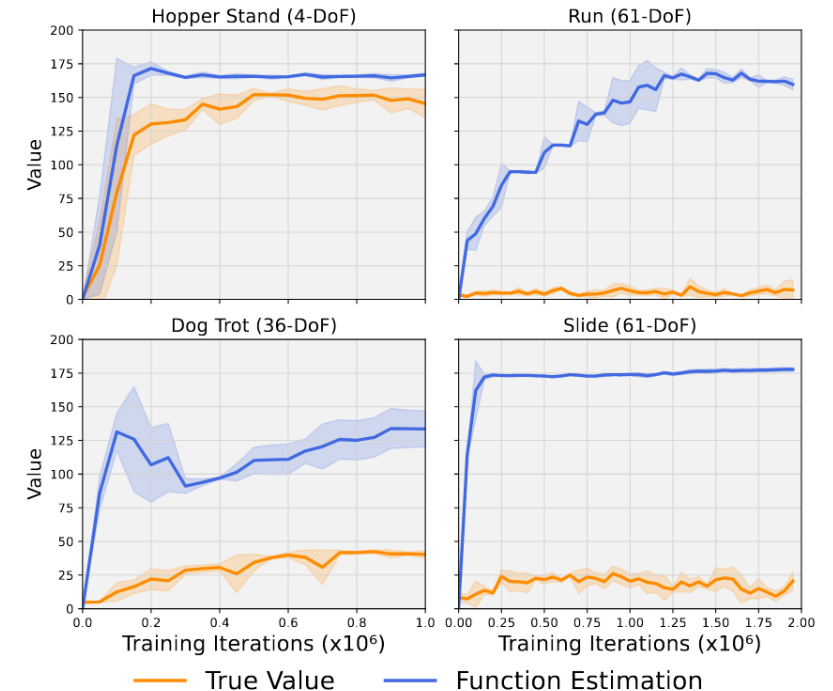
Policy mismatch를 극복하기 위한 개선

TD-MPC의 Policy mismatch 문제

- ✓ Terminal value 예측이 맞는지
 - ✓ $Q_\theta(z_i, a_i)$ 는 **planning policy**의 action
 - ✓ $Q_\theta(z_{i+1}, \pi_\theta(z_{i+1}))$ 은 **policy prior** p_θ 의 action
- ✓ **Policy Mismatch 발생!!**

$$\mathcal{L}(\theta; \Gamma_i) = c_1 \underbrace{\|R_\theta(z_i, a_i) - r_i\|_2^2}_{\text{reward}} + c_2 \underbrace{\|Q_\theta(z_i, a_i) - (r_i + \gamma Q_\theta(z_{i+1}, \pi_\theta(z_{i+1})))\|_2^2}_{\text{value}} + c_3 \underbrace{\|d_\theta(z_i, a_i) - h_\theta(s_{i+1})\|_2^2}_{\text{latent state consistency}}$$

- ✓ **Policy Mismatch**
 - ✓ 환경과 상호작용은 똑똑한 **Planner**(=behavior policy)가
 - ✓ Terminal value estimation은 상대적으로 멍청한 **policy prior**가 해서 생기는 문제
 - ✓ 상대적으로 안 좋은 State-Action pair를 policy prior 가 선택하더라도,
 - ✓ Planning 과정에서 교정되어, 좋은 reward를 얻을 수 있다.
 - ✓ Policy prior는 이걸 알 수 없으며,
 - ✓ 따라서 **over estimated**된 value를 가지게 된다.



TD-M(PC)2의 개선 방안 및 결과

✓ 개선 방안) Behavior policy 와 Policy prior 간의 D_{KL} 을 제약 (PPO)

✓ Buffer에서 본 적도 없는 state-action pair가 over estimated된 value를 가지지 않도록 억제

✓ Out-of-distribution query

$$\pi_{k+1} := \operatorname{argmax}_{\pi} \mathbb{E}_{a \sim \pi(\cdot|s)} [Q_k(a, s)]$$

$$\text{s.t. } \mathbf{D}_{\mathbf{KL}}(\pi_{k+1} || \mu_k) \leq \epsilon$$

$$\mathcal{L}_{\pi} = - \mathbb{E}_{a \sim \pi} [Q(s, a) - \alpha \log \pi(a|s) + \beta \log \mu(a|s)]$$

